EP

Exam Proctor

Project report

01 Project Overview

02 Problem Statement

03 Problems Solved

04 System Architecture

05 Data Flow Diagrams

06 Technology Stack

07 Database Schema

08 Security & Privacy

09 Implementation Status

Core pipeline implemented & verified

```
repo: exam-monitor
stack: Next.js·PostgreSQL·LiveKit
role model: Admin / Proctor / Student
```

ACADEMIC / TECHNICAL PROJECT REPORT

Exam Proctor

A web platform for live-proctored online examinations — students sit timed exams under fullscreen lockdown while behavioral signals and on-device AI vision are streamed to a human proctor in real time and persisted for post-exam review with a computed integrity score.

Stack — Next.js 16 (App Router) · React 19 · TypeScript · PostgreSQL + Prisma · Socket.IO · LiveKit (WebRTC SFU) · MediaPipe / TensorFlow.js · Tailwind CSS v4

01 Project Overview

Exam Proctor lets an institution run a timed exam online while giving a human proctor the same situational awareness they'd have walking a physical exam hall — who's on camera, who just tabbed away, who's being flagged by the system — without requiring a 1:1 human-to-candidate ratio.

The platform serves three roles against one shared session model: **Admin** builds and schedules exams and assigns proctors; **Proctor** live-monitors assigned sessions, receives alerts, and can warn or terminate a candidate; **Student** passes a device pre-check, then takes the exam inside a locked-down, fullscreen runtime with a server-authoritative countdown.

Every observation the system makes — a tab switch, a face leaving frame, a phone in view — becomes one typed, timestamped event. Each event is pushed to the proctor's live alert feed *and* written to the database, so the same event stream that drives real-time monitoring also reconstructs an exact post-exam timeline.

Live video

The proctor sees a real-time grid of every candidate's webcam feed, routed through a WebRTC SFU rather than a peer mesh.

Behavioral signals

Tab switches, window blur, fullscreen exit, copy/paste, right-click, DevTools heuristics, multi-monitor and network-loss detection.

On-device AI vision

Face presence/count, gaze direction, and phone/book detection run in the candidate's browser — no raw video leaves the device for this.

02 Problem Statement

Remote assessment removes the one thing a physical exam hall guarantees for free: a human watching. Existing answers to that gap each fail differently.

GAP 01

Manual proctoring doesn't scale

One invigilator per candidate on a video call is the only fully faithful remote replica of an exam hall — and it's economically impossible past a handful of concurrent candidates.

GAP 02

Fully-automated "AI proctoring" over-trusts a model

Auto-failing a candidate on a single low-confidence gaze or object detection produces false positives that a human would have dismissed in a second, and gives candidates no recourse.

GAP 03

Client-side timers and answers can't be trusted

An exam runtime that trusts the browser's clock or lets the client hold the answer key is trivially defeated by anyone with DevTools open.

GAP 04

Media and event traffic don't share one channel well

Continuous video is heavy and needs fan-out (an SFU); behavioral events are hundreds of tiny high-frequency messages that need sub-second delivery. Serverless request/response can't hold either open.

The design goal, concretely: build a system where **the server is the source of truth** for the timer, the answer key and the integrity score; where AI signals are **assistive evidence attached to a human review**

workflow, never an automatic verdict; and where the two very different kinds of real-time traffic — video and events — are routed on infrastructure suited to each.

03 Problems Solved

What the system above actually does, end to end, mapped to the gap it closes.

3.1 Trustworthy exam runtime

- **Server-authoritative timer** — remaining time is computed from `startedAt + durationMin` on every request, never read from the client clock; auto-submits at zero.
- **Fullscreen lockdown** — the exam enters fullscreen on start; exiting is logged as a `FULLSCREEN_EXIT` event and can be configured to warn-then-terminate.
- **Debounced autosave** — answers persist to `/api/sessions/:id/answers` as the candidate works; the correct answer key is never sent to the browser.

3.2 Scalable live monitoring

- **One LiveKit room per session** — the candidate is `canPublish`-only, the proctor is `canSubscribe`-only; tokens are minted server-side and the LiveKit secret never reaches the browser.
- **Grid subscribes on demand** — the proctor's tile grid pulls low-res streams and upgrades to high-res only on tile-expand, so a 20–30 candidate room doesn't melt the proctor's CPU or bandwidth.

3.3 Real-time, durable event pipeline

- **Dedicated Socket.IO service** (`realtime/`) separate from the Next.js request/response cycle, because serverless functions can't hold a long-lived socket open.
- **JWT-authenticated handshake** — room membership is derived from the database on connect, never trusted from client-supplied claims.
- **Batched persistence** — events are buffered and bulk-inserted roughly once a second, so a burst of behavioral events can't hammer Postgres with row-at-a-time writes.

3.4 Human-in-the-loop integrity scoring

- **Weighted 0–100 score** — starts at 100, subtracts per event by severity with a per-type cap (so a chatty `TAB_HIDDEN` run can't alone zero the score), recomputed on submit.
- **Below-threshold sessions auto-flag for review** — they don't auto-fail; a proctor or admin makes the call from the recording and event timeline.
- **Evidence, not verdicts** — a `HIGH/CRITICAL` event captures a snapshot as attached evidence rather than acting unilaterally.

3.5 Privacy-conscious AI vision

Face detection, gaze estimation and object detection (MediaPipe FaceLandmarker + TensorFlow.js COCO-SSD) all run *on the candidate's device*. Only a compact event — type, severity, optional confidence — leaves the browser; the continuous camera feed itself is never analyzed server-side, only optionally recorded via LiveKit Egress for human review.

EVENT TYPE	SEVERITY	DETECTOR
FACE_NONE	high	MediaPipe FaceLandmarker — no face >3s
FACE_MULTIPLE	critical	MediaPipe FaceLandmarker — ≥2 faces
LOOKING_AWAY	medium	Gaze / head-pose from face landmarks
PHONE_DETECTED	critical	TFJS COCO-SSD object detector
BOOK_DETECTED	high	TFJS COCO-SSD object detector
FULLSCREEN_EXIT	high	Fullscreen API — behavioral guard
TAB_HIDDEN	medium	Page Visibility API — behavioral guard
DEVTOOLS_OPEN	medium	Window-size / timing heuristic (best-effort)

04 System Architecture

Three independently-deployable pieces, split along the line that actually matters: media traffic versus event traffic.

Media channel — WebRTC via LiveKit SFU
Continuous audio/video, ~1–2 Mbps per candidate. Needs a Selective Forwarding Unit so one proctor can fan out to N candidates without a full mesh; recorded via server-side Egress to object storage.

Event channel — WebSocket via Socket.IO
Sub-kilobyte JSON messages, high frequency, low latency required. A dedicated long-lived Node service, decoupled from Next.js's request/response model, batches writes to protect Postgres.

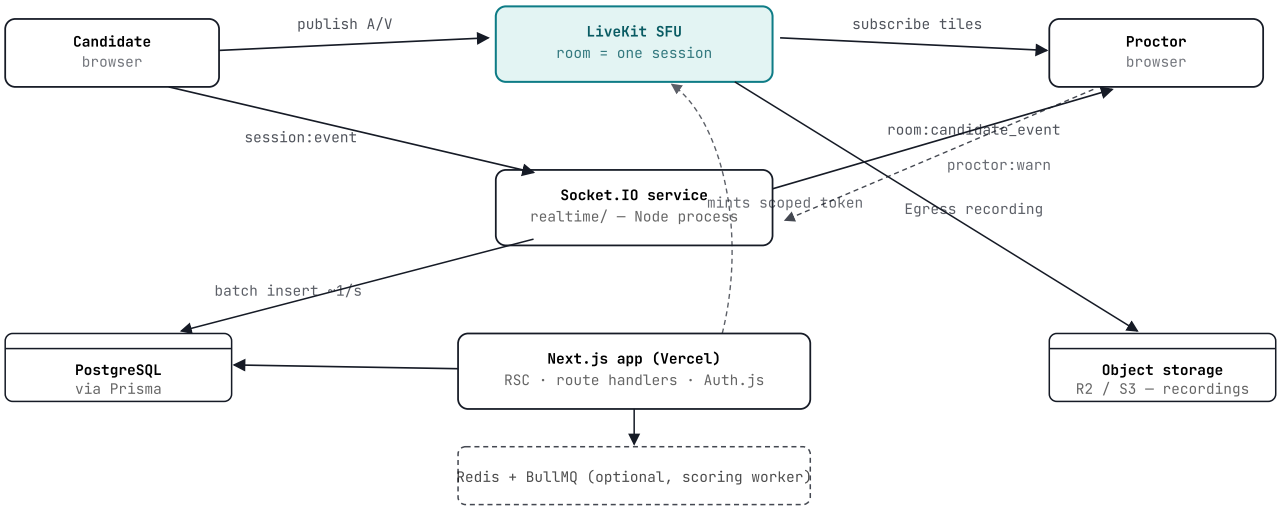


Figure 1. Media (top path, via LiveKit) and behavioral events (middle path, via Socket.IO) are kept on separate transports; the Next.js app is the shared source of truth for auth, session state and access tokens, and Redis/BullMQ is an optional accelerator since submit already scores inline.

05 Data Flow Diagrams

Standard DFD notation: rectangles are external entities, rounded rectangles are processes, open-ended bars are data stores.

5.1 Context diagram (Level 0)

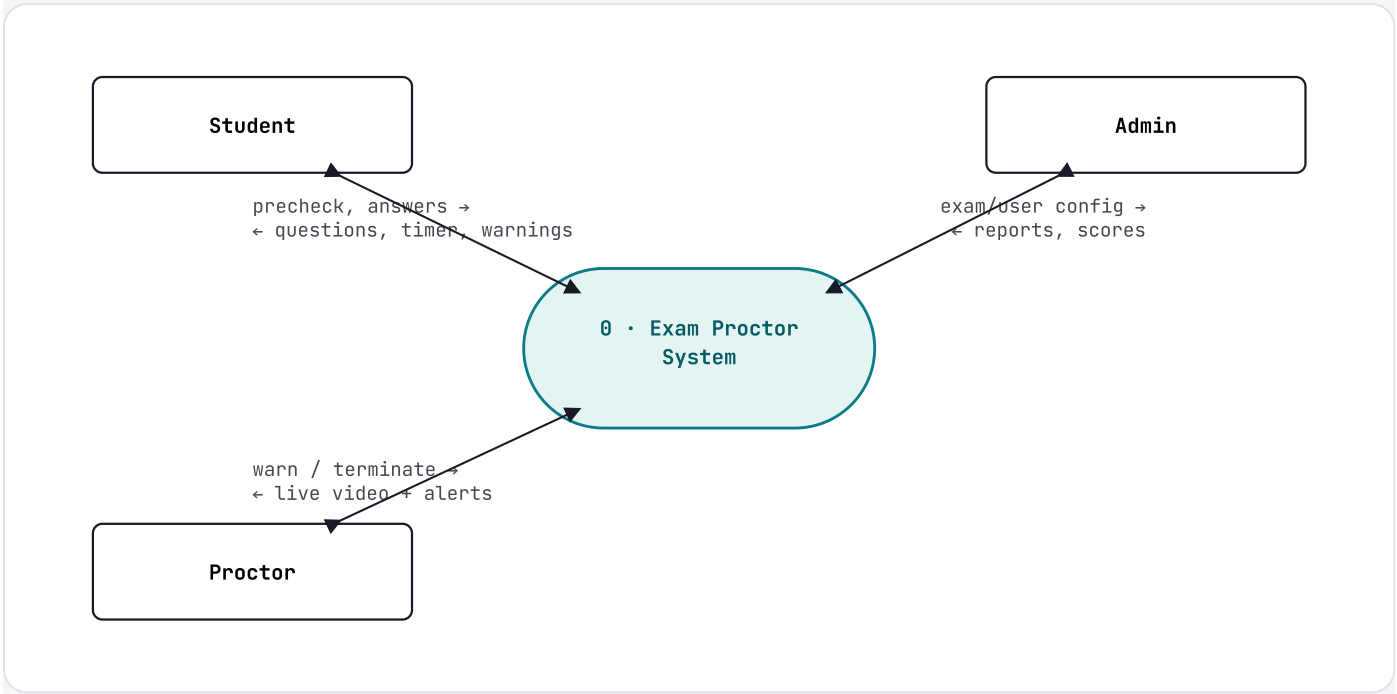


Figure 2. The system as a single black-box process; each external actor gets one two-way connection to it.

5.2 Level 1 — exam & monitoring pipeline

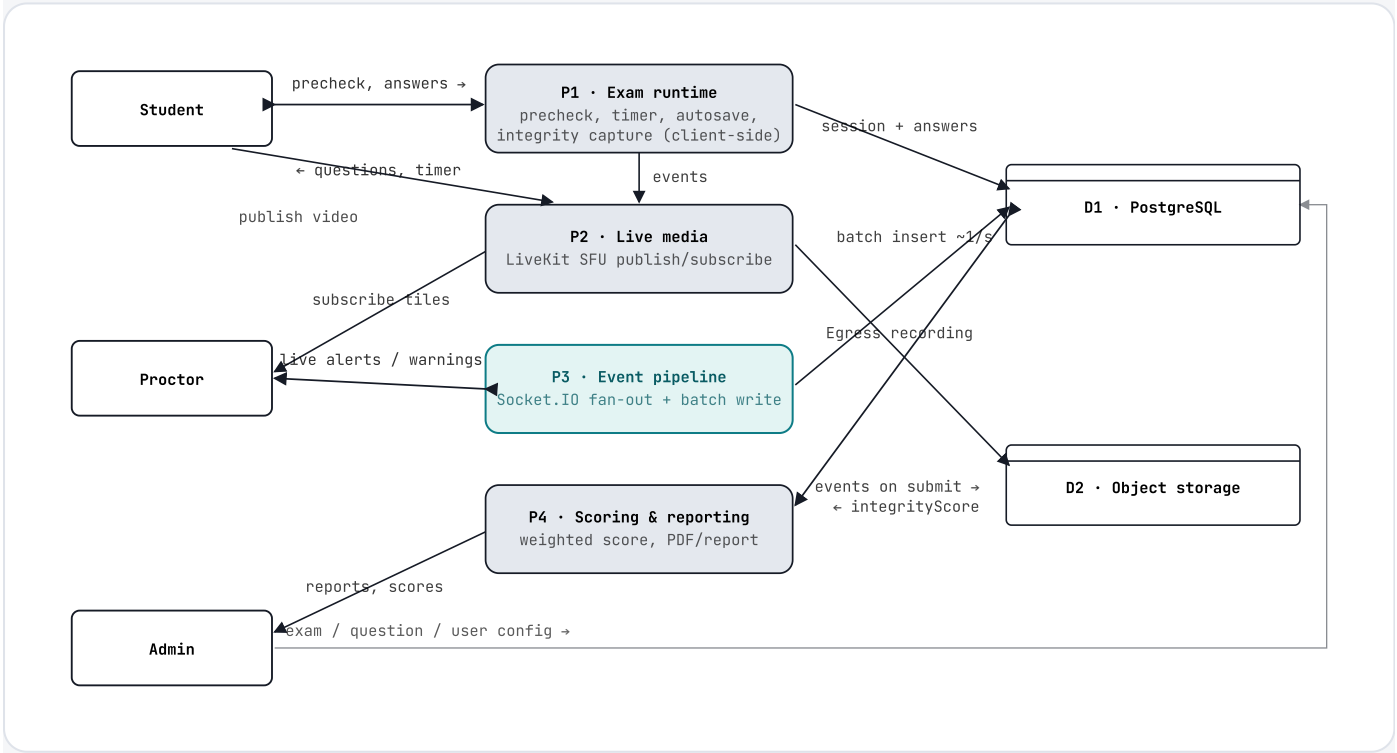


Figure 3. P1 owns integrity capture as part of the exam runtime — only compact event JSON, never raw video or DOM state, leaves it for P3. P4 reads events and answers from D1 on submit and writes the score straight back.

06 Technology Stack

As installed in `package.json` — chosen per concern, not as a single "framework of choice."

CONCERN	CHOICE	WHY THIS, HERE
Framework	`Next.js 16.3 (App Router) · React 19.2 · TypeScript`	RSC pages and Route Handlers cover the UI and the REST API from one repo.
Styling / UI	`Tailwind CSS v4 · Radix UI · class-variance-authority · lucide-react`	Accessible primitives, consistent variant-driven components.
Client state	`TanStack Query v5 · Zustand v5`	Query caches/mutates server data; Zustand drives the low-latency in-exam state machine (`examStore`).
Auth	`NextAuth v5 (beta) · @auth/prisma-adapter · argon2`	Credentials + JWT sessions carrying role claims; self-hosted, no third-party dependency for login.
Database / ORM	`PostgreSQL · Prisma 6.19`	Relational integrity for users/exams/sessions/events; types flow end-to-end into the app.
Object storage	`AWS SDK v3 (S3 client + presigner) → Cloudflare R2 / S3`	Recordings and AI-proctoring snapshot evidence, addressed by signed URL.
Live media	`LiveKit — livekit-client, livekit-server-sdk, @livekit/components-react`	SFU architecture: one proctor fans out to many candidates without a peer mesh; built-in Egress recording.
Realtime events	`Socket.IO 4.8 (server + client), standalone Node service`	Sub-second delivery for high-frequency tiny messages; runs outside the serverless request cycle.
AI vision (client)	`@mediapipe/tasks-vision (FaceLandmarker) · TensorFlow.js + COCO-SSD`	Face/gaze and object detection run on-device — private by construction, zero inference cost on the server.
Background jobs	`BullMQ · ioredis`	Optional accelerator for scoring recompute; submit scores inline either way.
Email	`Resend`	Transactional email for invites/warnings/reports.
Validation	`Zod`	One schema for API input, environment variables, and react-hook-form.
Deploy	`Vercel (web) · Railway (realtime) · LiveKit Cloud · Neon (Postgres)`	Each service deployed to the host suited to its runtime shape — see `railway.json`.

07 Database Schema

`prisma/schema.prisma` — six models, five enums, PostgreSQL.

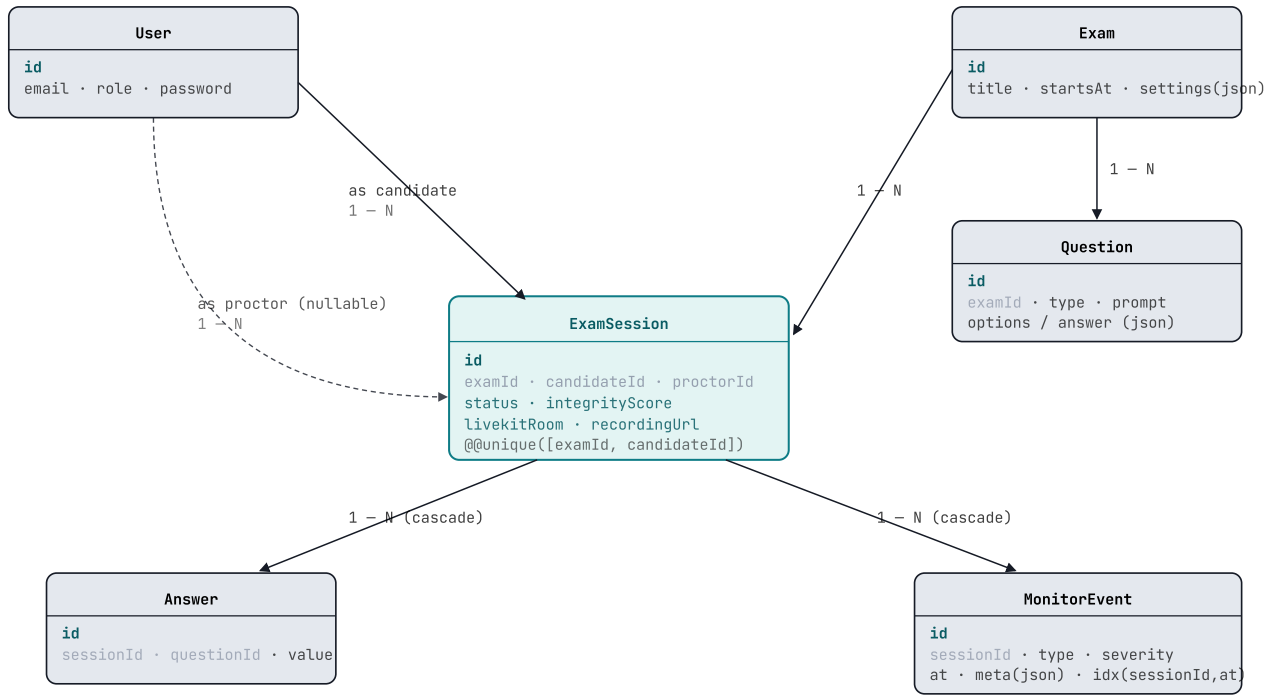


Figure 4. `ExamSession` is the join point of the whole model: one row per candidate per exam, referenced by every answer and every monitoring event it produced.

User	6 fields
id	String · cuid
email	String · unique
name	String
role	Role (ADMIN/PROCTOR/STU
password	String · argon2 has
createdAt	DateTime

Exam	8 fields
id	String · cuid
title / description	String / String
startsAt	DateTime
durationMin	Int
status	ExamStatus
settings	Json
createdBy	String

Question	8 fields
id	String · cuid
examId	FK → Exam, cascade
order	Int
type	String (MCQ/MULTI/SHOR
prompt	String
options / answer	Json? / Json? (never
points	Int · default 1

ExamSession	11 fields
id	String · cuid
examId / candidateId	FK → Exam / User
proctorId	FK → User, nullabl
status	SessionStatus
startedAt / submittedAt	DateTime?
integrityScore	Int? · 0–100
livekitRoom	String · unique
recordingUrl	String?

Answer	5 fields
id	String · cuid
sessionId	FK → ExamSession,
questionId	String
value	Json
updatedAt	DateTime · @updat

MonitorEvent	6 fields
id	String · cuid
sessionId	FK → ExamSession,
type	EventType · 21 values
severity	Severity
at	DateTime · indexed
meta	Json? (confidence, snap

7.1 Enums

SessionStatus

PENDING → PRECHECK → IN_PROGRESS → SUBMITTED / FLAGGED / TERMINATED

Severity

INFO · LOW · MEDIUM · HIGH · CRITICAL — weights integrity scoring

08 Security & Privacy

LEAST-PRIVILEGE MEDIA

Scoped LiveKit tokens

Minted server-side per session; candidates get `canPublish`, proctors get `canSubscribe`. The LiveKit API secret never reaches the browser.

SERVER AS SOURCE OF TRUTH

Timer, answer key, score

All computed server-side. The exam runtime never receives correct answers; the client only ever reports events.

AUTHORIZATION

Row-level checks on every handler

A candidate can read/write only their own session; a proctor only sessions for exams they're assigned to. Never trusted from client claims.

DATA MINIMIZATION

On-device inference

Face/gaze/object detection runs in-browser; only a compact typed event (plus an optional evidence snapshot on HIGH+ events) leaves the device.

ASSISTIVE, NOT AUTOMATIC

Human review on flag

AI signals feed a weighted score and a flag threshold — never an auto-fail. A proctor/admin makes the final call from the recording and timeline.

GRACEFUL DEGRADATION

Optional integrations fail safe

LiveKit / R2 / Resend routes return a clear 503 when unconfigured instead of crashing — core auth, exam CRUD and the student/admin flows stay usable without them.

09 Implementation Status

Per the project's own README — what's verified end-to-end versus simplified from the original spec.

Implemented & verified

Auth + RBAC (admin/proctor/student) · exam & question CRUD · student exam runtime (server timer, autosave, autosubmit) · Socket.IO event pipeline (guard hook → fan-out → batched persistence → live alert feed) · proctor warn/terminate · integrity scoring · post-exam report & timeline.

Simplified from the original spec

Single-repo layout instead of a monorepo — the realtime service shares Prisma/types via relative imports. Proctor grid subscribes to video only on tile-expand. BullMQ recompute is optional; submit already scores inline.